Шпаргалка: SVM primal (прямая постановка)

Идея. Найти линию `w·x + b = 0`, которая делит классы и оставляет самый широкий зазор `2/‖w‖`. Ищем `w` и `b` градиентным спуском.

1. Функция потерь

$$L = \|\mathbf{w}\|^2 + \frac{1}{N}\sum_{i=1}^{N} \max\bigl(0,\ 1 - y_i(\mathbf{w}\cdot x_i + b)\bigr)$$

- $\|\mathbf{w}\|^2$ — хотим широкий зазор (маленький $\mathbf{w}$);
- второе слагаемое — **hinge loss**: хотим, чтобы у каждой точки запас $m_i = y_i(\mathbf{w}\cdot x_i + b)$ был ≥ 1;
- метки: строго $y \in \{-1, +1\}$ (из 0/1 переводим: 0 → −1);
- само значение L не важно, важно, при каких $\mathbf{w}$ и b оно минимально.

Запас m_i	**Где точка**	**Штраф**
$m_i \geq 1$	вне зазора, на своей стороне	0
$0 < m_i < 1$	внутри зазора	$1 - m_i$
$m_i \leq 0$	на чужой стороне	$1 - m_i$ (большой)

2. Градиент для одной точки

Hinge ломается в $m = 1$, поэтому берём производную по веткам (субградиент).

Условие	$\nabla_{\mathbf{w}}$	∇_b
$y_i(\mathbf{w}\cdot x_i + b) \geq 1$	$2\mathbf{w}$	0
$y_i(\mathbf{w}\cdot x_i + b) < 1$	$2\mathbf{w} - y_i x_i$	$-y_i$

Шаг: $\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla_{\mathbf{w}}$, $\quad b \leftarrow b - \alpha\nabla_b$ (α — learning rate).

3. Алгоритм

```
1. метки 0/1 -> -1/+1
2. w = [1, 1, ..., 1],  b = 1
3. повторить n_iter раз:
       для каждой точки (x_i, y_i):
           m = y_i * (w · x_i + b)
           если m >= 1:  grad_w = 2*w;            grad_b = 0
           иначе:        grad_w = 2*w - y_i*x_i;  grad_b = -y_i
           w = w - alpha * grad_w
           b = b - alpha * grad_b
       (для лога) L = w·w + среднее(max(0, 1 - y*(w·x + b)))
4. вернуть w, b
```

Предсказание: `w · x + b > 0` → класс 1, иначе → класс 0.

4. Что помнить

- Обновление идёт **по одной точке**, поэтому $1/N$ в шаге нет (оно только в L для лога). L в целом падает, но не на каждом шаге.
- Градиент $-y_i x_i$ «подталкивает» плохую точку: класс +1 тянет $\mathbf{w}$ к точке, класс −1 отталкивает.
- $\|\mathbf{w}\|^2$ без $\frac{1}{2}$ (как в коде): минимум тот же, производная $2\mathbf{w}$. Квадрат вместо $\|\mathbf{w}\|$ — без корня считать проще, ответ тот же.
- α большое — линия прыгает; маленькое — медленно. Признаки лучше масштабировать.
- Параметр `C` (если добавлен в код) умножает hinge: больше `C` — строже к ошибкам, зазор уже.
- Primal работает прямо с $\mathbf{w}$ и b (быстро, когда признаков немного). Ядра возможны только в **dual** (часть 3).